

Week 5: Population Projections

SOC6708 ADA

Monica Alexander

For this lab, we are going to pick two countries from the WPP data (one high fertility, one low fertility) and do some projections and calculations.

Load in data

```
library(tidyverse)
library(here)
```

Loading in the data. For this lab we will need fertility rates, populations by age, and the Lx columns of the life table.

```
# Read and select variables for age-specific fertility rates
df <- read_csv(here("data/WPP2024_Fertility_by_Age5.csv"))
df <- df |>
  select(Location, Time, ASFR, AgeGrp)

# Read and select female life table survivorship columns
dl <- read_csv(here("data/WPP2024_Life_Table_Abridged_Medium_1950-2023.csv"))
dl <- dl |>
  filter(Variant=="Medium", Sex=="Female")|>
  select(Location, Time, AgeGrp,Lx)

# Read female population counts, reshape to long format, and calculate single-year ages
d_female <- read_csv(here("data/wpp_pop_clean.csv")) |>
  filter(sex=="Female")
d_female <- d_female |>
  pivot_longer(x0:x100) |>
  mutate(age = as.numeric(str_remove(name, "x")))
```

```
# Construct 5-year age group brackets to match life tables and fertility data
breaks <- c(seq(0,100, 5), Inf)
labels <- c("0-4", "5-9", "10-14", "15-19", "20-24", "25-29", "30-34", "35-39",
           "40-44", "45-49", "50-54", "55-59", "60-64", "65-69", "70-74",
           "75-79", "80-84", "85-89",
           "90-94", "95-99", "100+")

d_female$age_group <- cut(d_female$age, breaks = breaks, labels = labels, right = FALSE)
```

Population projection

1. pick two countries, 1 with high fertility and 1 with low fertility. Here are the options:

```
# unique(df$Location)
# excluding as it prints a long list in PDF
# monica to do australia and zimbabwe
```

2. Calculate the NRR for each of the countries in 2020. Confirm that these are one high and one low, based on NRR values.

```
# Calculate 2020 discrete Net Reproduction Rate (NRR) for Australia and Zimbabwe
df |>
  filter(Location == "Australia"|Location == "Zimbabwe", Time == 2020) |>
  left_join(dl) |>
  mutate(prod = ASFR/1000*Lx/10^5) |>
  group_by(Location) |>
  summarize(NRR = sum(prod)*0.4886)
```

```
# A tibble: 2 x 2
  Location    NRR
  <chr>      <dbl>
1 Australia 0.775
2 Zimbabwe  1.67
```

Remember: discrete version of NRR is

$$NRR = \sum_x {}_nL_x \cdot {}_nF_x \cdot f_{fab}$$

and the fraction female at birth is something like 0.4886.

Leslie matrix

3. Create a Leslie matrix for each of your countries based on 2020 rates. The function below will help. This is pretty ugly so if someone want to rewrite it feel free :)

Note: Fx only covers 10-50, but it needs to be 0 for all other age groups (probably easiest to `left_join df` to `d1` then replace the NA values with 0s). Note 2: From the WPP data we have a split first age group, but want to convert this into 0-5. Just sum the Lx values to get ${}_5L_0$

```
# Function to construct a Leslie Matrix using survivorship and fertility profiles
leslie <- function(nLx,
                  nFx,
                  n_age_groups=21, # based on 0 to 100+
                  ffab = 0.4886){
  L = matrix(0, nrow = n_age_groups, ncol = n_age_groups)
  L[1,] = ffab * nLx[1]*(nFx[1:n_age_groups]+nFx[2:(n_age_groups+1)]*nLx[2:(n_age_groups+1)])
  L[1,ncol(L)] <- 0
  diag(L[2:n_age_groups,1:(n_age_groups-1)]) = nLx[2:n_age_groups] / nLx[1:(n_age_groups-1)]
  return(L)
}
```

```
# Prepare Australia's 2020 demographic data and construct its Leslie matrix
d_austr <- d1 |>
  filter(Location == "Australia", Time == 2020) |>
  left_join(df) |>
  mutate(ASFR = ifelse(is.na(ASFR), 0, ASFR)) |>
  mutate(age_group = ifelse(AgeGrp=="0"|AgeGrp=="1-4", "0-5", AgeGrp)) |>
  group_by(age_group) |>
  summarize(Lx = sum(Lx/10^5), Fx = sum(ASFR/1000)) |>
  mutate(age = as.numeric(str_remove(age_group, "-.*"))) |>
  mutate(age = ifelse(is.na(age), 100, age)) |>
  arrange(age)
```

```
Fx_austr <- d_austr$Fx
```

```
Lx_austr <- d_austr$Lx
```

```
A_austr <- leslie(nLx = Lx_austr, nFx = Fx_austr)
```

```
A_austr
```

```
      [,1]      [,2]      [,3]      [,4]      [,5]      [,6]
[1,] 0.0000000 7.305526e-05 0.009476387 0.05510588 0.1433688 0.2324438
```

[2,]	0.9995289	0.000000e+00	0.000000000	0.000000000	0.0000000	0.00000000	
[3,]	0.0000000	9.997150e-01	0.000000000	0.000000000	0.0000000	0.00000000	
[4,]	0.0000000	0.000000e+00	0.999314286	0.000000000	0.0000000	0.00000000	
[5,]	0.0000000	0.000000e+00	0.000000000	0.99886464	0.0000000	0.00000000	
[6,]	0.0000000	0.000000e+00	0.000000000	0.000000000	0.9988247	0.00000000	
[7,]	0.0000000	0.000000e+00	0.000000000	0.000000000	0.0000000	0.9983734	
[8,]	0.0000000	0.000000e+00	0.000000000	0.000000000	0.0000000	0.00000000	
[9,]	0.0000000	0.000000e+00	0.000000000	0.000000000	0.0000000	0.00000000	
[10,]	0.0000000	0.000000e+00	0.000000000	0.000000000	0.0000000	0.00000000	
[11,]	0.0000000	0.000000e+00	0.000000000	0.000000000	0.0000000	0.00000000	
[12,]	0.0000000	0.000000e+00	0.000000000	0.000000000	0.0000000	0.00000000	
[13,]	0.0000000	0.000000e+00	0.000000000	0.000000000	0.0000000	0.00000000	
[14,]	0.0000000	0.000000e+00	0.000000000	0.000000000	0.0000000	0.00000000	
[15,]	0.0000000	0.000000e+00	0.000000000	0.000000000	0.0000000	0.00000000	
[16,]	0.0000000	0.000000e+00	0.000000000	0.000000000	0.0000000	0.00000000	
[17,]	0.0000000	0.000000e+00	0.000000000	0.000000000	0.0000000	0.00000000	
[18,]	0.0000000	0.000000e+00	0.000000000	0.000000000	0.0000000	0.00000000	
[19,]	0.0000000	0.000000e+00	0.000000000	0.000000000	0.0000000	0.00000000	
[20,]	0.0000000	0.000000e+00	0.000000000	0.000000000	0.0000000	0.00000000	
[21,]	0.0000000	0.000000e+00	0.000000000	0.000000000	0.0000000	0.00000000	
	[,7]	[,8]	[,9]	[,10]	[,11]	[,12]	[,13]
[1,]	0.2164771	0.1003821	0.02018435	0.00148588	0.0000000	0.0000000	0.00000000
[2,]	0.0000000	0.0000000	0.000000000	0.000000000	0.0000000	0.0000000	0.00000000
[3,]	0.0000000	0.0000000	0.000000000	0.000000000	0.0000000	0.0000000	0.00000000
[4,]	0.0000000	0.0000000	0.000000000	0.000000000	0.0000000	0.0000000	0.00000000
[5,]	0.0000000	0.0000000	0.000000000	0.000000000	0.0000000	0.0000000	0.00000000
[6,]	0.0000000	0.0000000	0.000000000	0.000000000	0.0000000	0.0000000	0.00000000
[7,]	0.0000000	0.0000000	0.000000000	0.000000000	0.0000000	0.0000000	0.00000000
[8,]	0.9977068	0.0000000	0.000000000	0.000000000	0.0000000	0.0000000	0.00000000
[9,]	0.0000000	0.9967387	0.000000000	0.000000000	0.0000000	0.0000000	0.00000000
[10,]	0.0000000	0.0000000	0.99475599	0.000000000	0.0000000	0.0000000	0.00000000
[11,]	0.0000000	0.0000000	0.000000000	0.99201745	0.0000000	0.0000000	0.00000000
[12,]	0.0000000	0.0000000	0.000000000	0.000000000	0.9881986	0.0000000	0.00000000
[13,]	0.0000000	0.0000000	0.000000000	0.000000000	0.0000000	0.9826185	0.00000000
[14,]	0.0000000	0.0000000	0.000000000	0.000000000	0.0000000	0.0000000	0.9741496
[15,]	0.0000000	0.0000000	0.000000000	0.000000000	0.0000000	0.0000000	0.00000000
[16,]	0.0000000	0.0000000	0.000000000	0.000000000	0.0000000	0.0000000	0.00000000
[17,]	0.0000000	0.0000000	0.000000000	0.000000000	0.0000000	0.0000000	0.00000000
[18,]	0.0000000	0.0000000	0.000000000	0.000000000	0.0000000	0.0000000	0.00000000
[19,]	0.0000000	0.0000000	0.000000000	0.000000000	0.0000000	0.0000000	0.00000000
[20,]	0.0000000	0.0000000	0.000000000	0.000000000	0.0000000	0.0000000	0.00000000
[21,]	0.0000000	0.0000000	0.000000000	0.000000000	0.0000000	0.0000000	0.00000000
	[,14]	[,15]	[,16]	[,17]	[,18]	[,19]	[,20]

[1,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
[2,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
[3,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
[4,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
[5,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
[6,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
[7,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
[8,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
[9,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
[10,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
[11,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
[12,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
[13,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
[14,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
[15,]	0.9587412	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
[16,]	0.0000000	0.9289526	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
[17,]	0.0000000	0.0000000	0.8708727	0.0000000	0.0000000	0.0000000	0.0000000
[18,]	0.0000000	0.0000000	0.0000000	0.7610938	0.0000000	0.0000000	0.0000000
[19,]	0.0000000	0.0000000	0.0000000	0.0000000	0.5829275	0.0000000	0.0000000
[20,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.3664131	0.0000000
[21,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.1949668

[,21]

[1,]	0
[2,]	0
[3,]	0
[4,]	0
[5,]	0
[6,]	0
[7,]	0
[8,]	0
[9,]	0
[10,]	0
[11,]	0
[12,]	0
[13,]	0
[14,]	0
[15,]	0
[16,]	0
[17,]	0
[18,]	0
[19,]	0
[20,]	0
[21,]	0

```
# Prepare Zimbabwe's 2020 demographic data and construct its Leslie matrix
d_zim <- dl |>
  filter(Location == "Zimbabwe", Time == 2020) |>
  left_join(df) |>
  mutate(ASFR = ifelse(is.na(ASFR), 0, ASFR)) |>
  mutate(age_group = ifelse(AgeGrp=="0"|AgeGrp=="1-4", "0-5", AgeGrp)) |>
  group_by(age_group) |>
  summarize(Lx = sum(Lx/10^5), Fx = sum(ASFR/1000)) |>
  mutate(age = as.numeric(str_remove(age_group, "-.*"))) |>
  mutate(age = ifelse(is.na(age), 100, age)) |>
  arrange(age)

Fx_zim <- d_zim$Fx
Lx_zim <- d_zim$Lx

A_zim <- leslie(nLx = Lx_zim, nFx = Fx_zim)

A_zim
```

	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]	[,7]
[1,]	0.0000000	0.001703963	0.1171986	0.3342485	0.4225261	0.3698402	0.2819795
[2,]	0.9883959	0.000000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
[3,]	0.0000000	0.995369780	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
[4,]	0.0000000	0.000000000	0.9939179	0.0000000	0.0000000	0.0000000	0.0000000
[5,]	0.0000000	0.000000000	0.0000000	0.9894125	0.0000000	0.0000000	0.0000000
[6,]	0.0000000	0.000000000	0.0000000	0.0000000	0.9851054	0.0000000	0.0000000
[7,]	0.0000000	0.000000000	0.0000000	0.0000000	0.0000000	0.9799538	0.0000000
[8,]	0.0000000	0.000000000	0.0000000	0.0000000	0.0000000	0.0000000	0.9724266
[9,]	0.0000000	0.000000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
[10,]	0.0000000	0.000000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
[11,]	0.0000000	0.000000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
[12,]	0.0000000	0.000000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
[13,]	0.0000000	0.000000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
[14,]	0.0000000	0.000000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
[15,]	0.0000000	0.000000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
[16,]	0.0000000	0.000000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
[17,]	0.0000000	0.000000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
[18,]	0.0000000	0.000000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
[19,]	0.0000000	0.000000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
[20,]	0.0000000	0.000000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
[21,]	0.0000000	0.000000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
	[,8]	[,9]	[,10]	[,11]	[,12]	[,13]	

[1,]	0.1613842	0.05294465	0.007632158	0.000218539	0.0000000	0.0000000	
[2,]	0.0000000	0.00000000	0.000000000	0.000000000	0.0000000	0.0000000	
[3,]	0.0000000	0.00000000	0.000000000	0.000000000	0.0000000	0.0000000	
[4,]	0.0000000	0.00000000	0.000000000	0.000000000	0.0000000	0.0000000	
[5,]	0.0000000	0.00000000	0.000000000	0.000000000	0.0000000	0.0000000	
[6,]	0.0000000	0.00000000	0.000000000	0.000000000	0.0000000	0.0000000	
[7,]	0.0000000	0.00000000	0.000000000	0.000000000	0.0000000	0.0000000	
[8,]	0.0000000	0.00000000	0.000000000	0.000000000	0.0000000	0.0000000	
[9,]	0.9634468	0.00000000	0.000000000	0.000000000	0.0000000	0.0000000	
[10,]	0.0000000	0.95341623	0.000000000	0.000000000	0.0000000	0.0000000	
[11,]	0.0000000	0.00000000	0.939678203	0.000000000	0.0000000	0.0000000	
[12,]	0.0000000	0.00000000	0.000000000	0.921973110	0.0000000	0.0000000	
[13,]	0.0000000	0.00000000	0.000000000	0.000000000	0.8954917	0.0000000	
[14,]	0.0000000	0.00000000	0.000000000	0.000000000	0.0000000	0.8543877	
[15,]	0.0000000	0.00000000	0.000000000	0.000000000	0.0000000	0.0000000	
[16,]	0.0000000	0.00000000	0.000000000	0.000000000	0.0000000	0.0000000	
[17,]	0.0000000	0.00000000	0.000000000	0.000000000	0.0000000	0.0000000	
[18,]	0.0000000	0.00000000	0.000000000	0.000000000	0.0000000	0.0000000	
[19,]	0.0000000	0.00000000	0.000000000	0.000000000	0.0000000	0.0000000	
[20,]	0.0000000	0.00000000	0.000000000	0.000000000	0.0000000	0.0000000	
[21,]	0.0000000	0.00000000	0.000000000	0.000000000	0.0000000	0.0000000	
	[,14]	[,15]	[,16]	[,17]	[,18]	[,19]	[,20] [,21]
[1,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000 0
[2,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000 0
[3,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000 0
[4,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000 0
[5,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000 0
[6,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000 0
[7,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000 0
[8,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000 0
[9,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000 0
[10,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000 0
[11,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000 0
[12,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000 0
[13,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000 0
[14,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000 0
[15,]	0.8031034	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000 0
[16,]	0.0000000	0.7357458	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000 0
[17,]	0.0000000	0.0000000	0.649318	0.0000000	0.0000000	0.0000000	0.0000000 0
[18,]	0.0000000	0.0000000	0.0000000	0.4867111	0.0000000	0.0000000	0.0000000 0
[19,]	0.0000000	0.0000000	0.0000000	0.0000000	0.3151101	0.0000000	0.0000000 0
[20,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.1892762	0.0000000 0
[21,]	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.102173 0

4. Confirm that the NRRs from the Leslie matrices are the same as your NRRs above. The formula is:

$$NRR = \sum A_{1,j(x)} \frac{{}_nL_x}{{}_nL_0}$$

where A is the Leslie matrix. I think the easiest way to do this in practice is `A[1,]%*%cumprod(c(1,diag(A[-1,])))`.

```
# Verify NRR values calculated from the Leslie matrices
A_aus[1,]%*%cumprod(c(1,diag(A_aus[-1,])))
```

```
      [,1]
[1,] 0.7754972
```

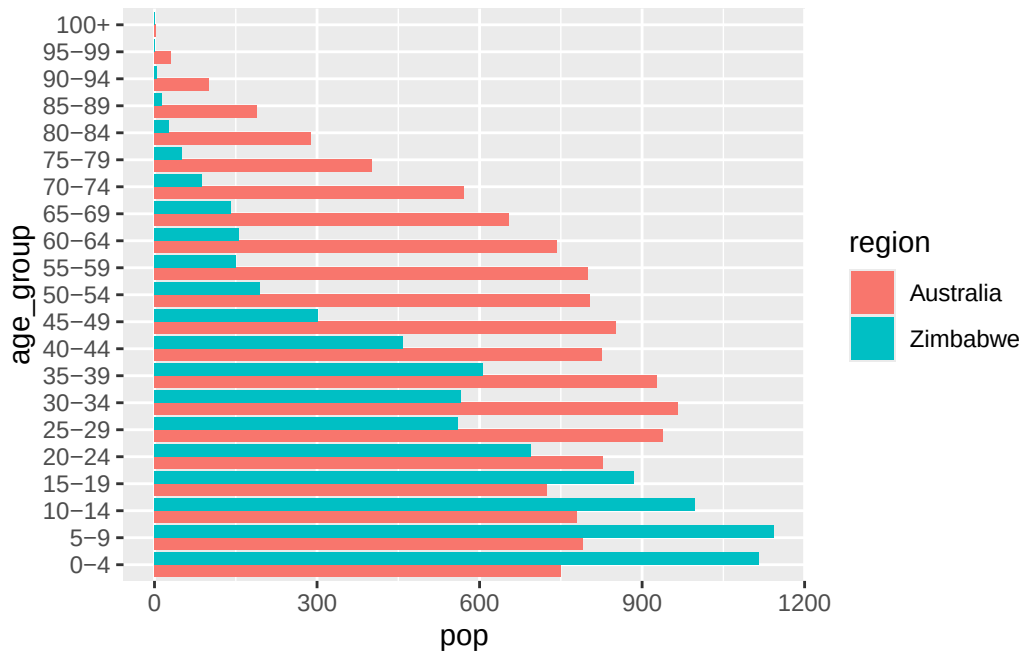
```
A_zim[1,]%*%cumprod(c(1,diag(A_zim[-1,])))
```

```
      [,1]
[1,] 1.667895
```

5. Get the 2020 populations for your two chosen countries. What does the age distribution look like (plot half an age pyramid). Given the NRRs what do expect the age pyramids ending up to look like?

```
# Aggregate and plot the 2020 female age distributions (half-pyramid)
dpop <- d_female |>
  filter(region == "Australia"|region == "Zimbabwe", year == 2020) |>
  group_by(region, age_group) |>
  summarize(pop = sum(value))

dpop |>
  ggplot(aes(age_group, pop, fill = region)) +
  geom_bar(stat = "identity", position = "dodge")+
  coord_flip()
```



6. Project each population forward 200 years (i.e. $200/5 = 40$ projection steps). For each country, you'll want to save every population for each age group and projection step in a matrix, and put the first column as your starting population. So something like this:

```
# Project the female populations forward 200 years (40 steps of 5-year intervals)
n <- 5
age_groups <- seq(0, 100, by = n)
n_age_groups <- length(age_groups)
n_projections <- 200/n

# Australia projection loop
initial_pop <- dpop |> filter(region=="Australia") |> select(pop) |> pull()
K <- matrix(0, nrow = n_age_groups, ncol = n_projections+1)
K[,1] <- initial_pop
for(i in 2:(n_projections+1)){
  this_projection <- A_aus%*%K[,i-1]
  K[,i] <- this_projection
}

# Zimbabwe projection loop
K_zim <- matrix(0, nrow = n_age_groups, ncol = n_projections+1)
K_zim[,1] <- dpop |> filter(region=="Zimbabwe") |> select(pop) |> pull()
for(i in 2:(n_projections+1)){
```

```

this_projection <- A_zim%%K_zim[,i-1]
K_zim[,i] <- this_projection
}

```

For the next bits, you probably want to take these two matrices and convert them back into a tibble for plotting. Something like this:

```

# Convert projection matrices back to long format and compute age-group proportions
Kdf <- as_tibble(K, .name_repair = "minimal")
colnames(Kdf) <- seq(from = 2020, to = (2020 + n_projections * n), by = 5)
Kdf <- cbind(age = seq(from = 0, to = 100, by = 5), Kdf)

Kdf_long <- Kdf |>
  pivot_longer(-age, names_to = "year", values_to = "pop") |>
  group_by(year) |>
  mutate(prop = pop / sum(pop))

# do the same for country 2

# then join them together

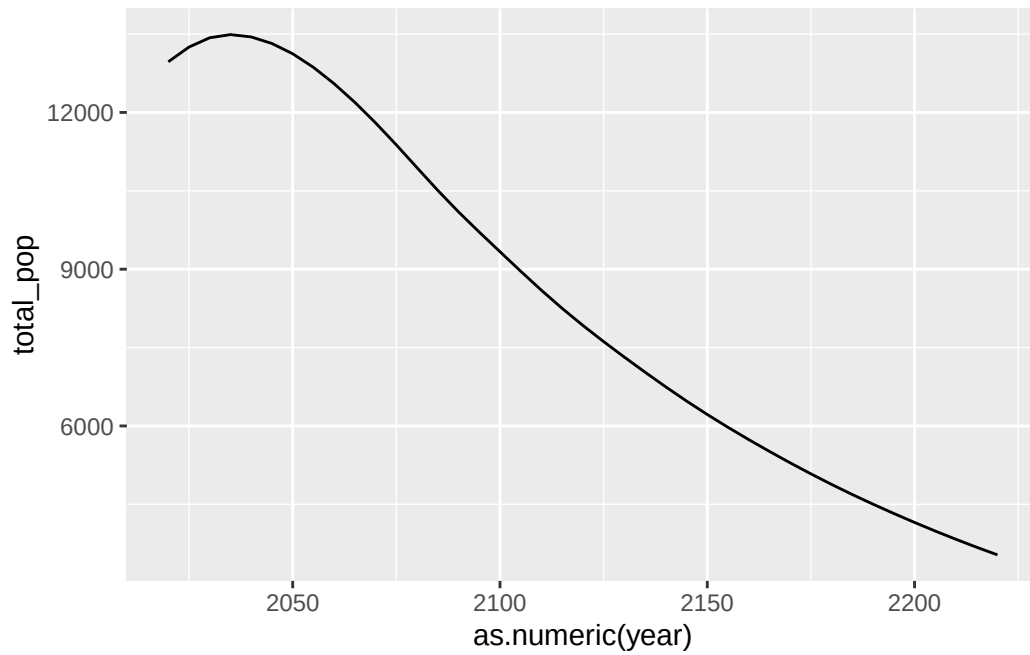
```

7a. Plot the total population size over time

```

# Plot total projected population size over time
Kdf_long |>
  group_by(year) |>
  summarize(total_pop = sum(pop)) |>
  ggplot(aes(as.numeric(year), total_pop)) +
  geom_line()

```



7b. Plot the proportion in each age group over time

```
# Plot the proportion of the population in each age group over time
Kdf_long |>
  ggplot(aes(as.numeric(year), prop, color = age, group = age)) +
  geom_line()
```

